MiniTransformer - Transformer for NLP

Part 1

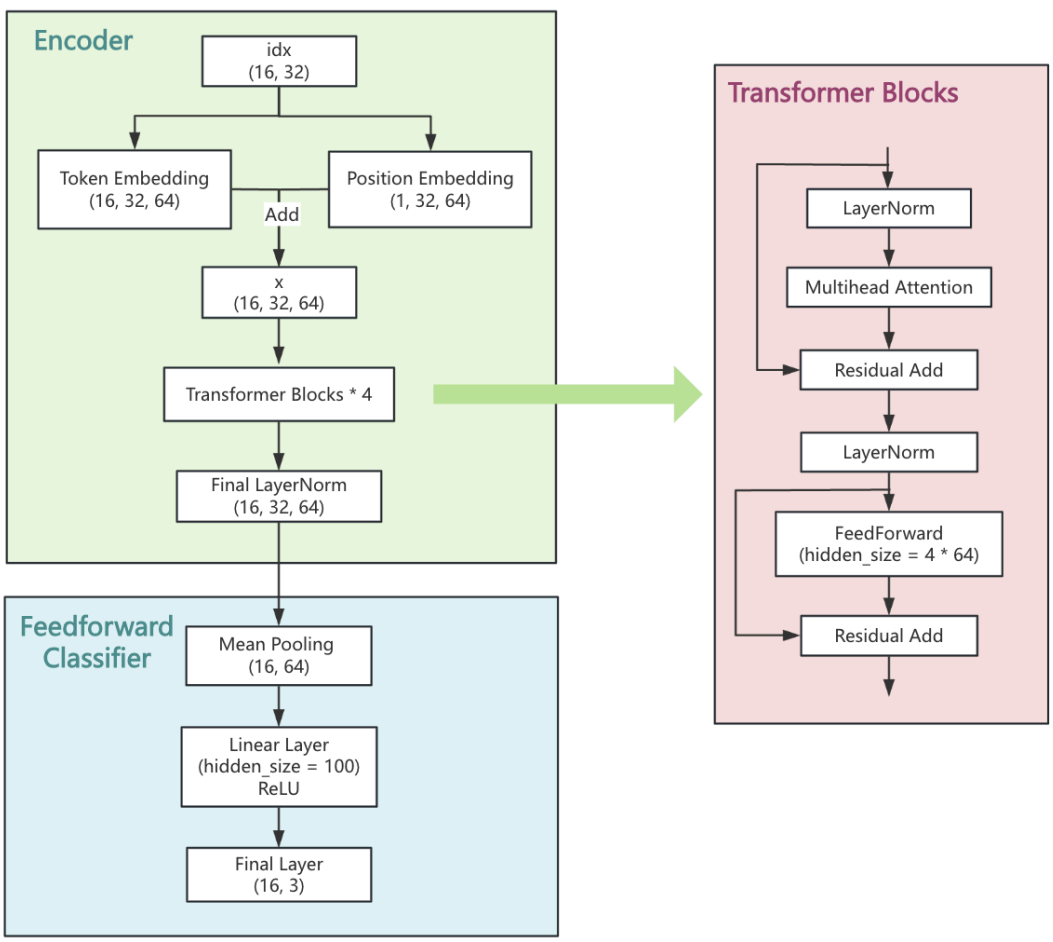
1.1 Encoder Implementation

The Transformer Encoder consists of a token embedding layer and a simple absolute positional embedding layer, both with dimension n_embd=64. The encoder comprises 4 stacked TransformerBlock layers, each containing a multi-head self-attention mechanism with 2 attention heads and a feedforward network with a 4* expansion factor. Each transformer block applies layer normalization before the attention and feedforward sub-layers, followed by residual connections. The encoder returns both the contextualized token embeddings and the attention weights from each layer for visualization purposes.

1.2 Feedforward Classifier Implementation

The classifier is a simple feedforward neural network with one hidden layer. It takes the average of embeddings in a sequence as input. The network first projects the 64-dimensional input to a hidden layer of size 100 using a linear transformation, applies a ReLU activation function, and then projects to the 3 output classes.

The figure below presents the architecture of the Encoder and the Transformer classifier. All hyperparameters are configured using default settings.

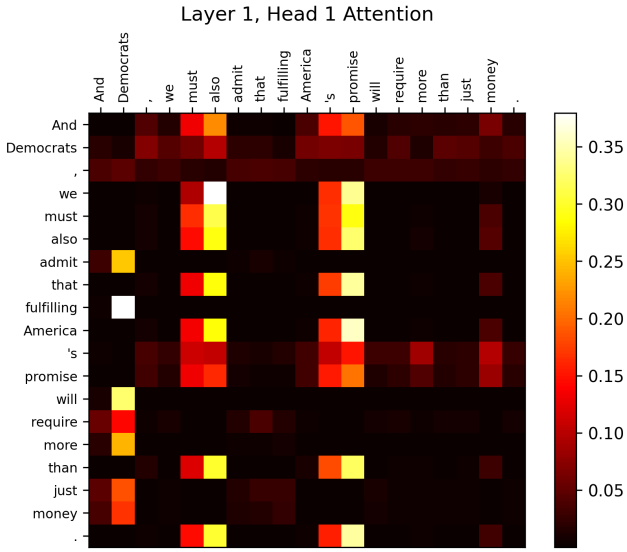


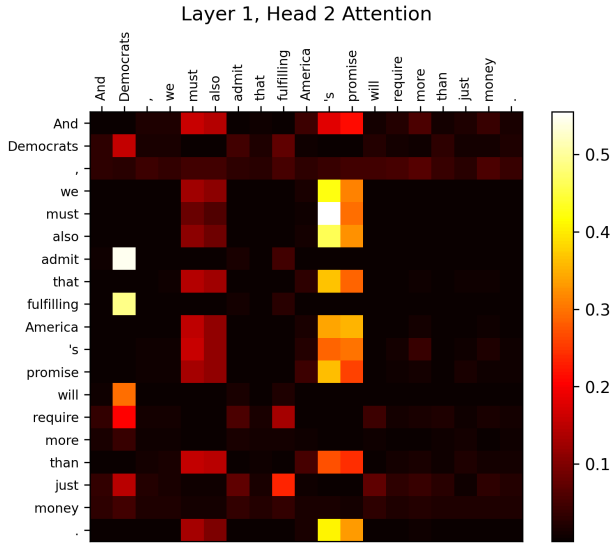
1.3 Joint Encoder & Classifier Training

The encoder and classifier are trained end-to-end using supervised learning on the train_CLS dataset. For each training batch, input token sequences are passed through the encoder to obtain contextualized embeddings, which are then averaged across the sequence dimension and fed into the classifier. The classifier outputs logits for the three politician classes. Cross-entropy loss is computed and backpropagated through both the classifier and the encoder. Training runs for 15 epochs using the Adam optimizer with a learning rate of 1e-3.

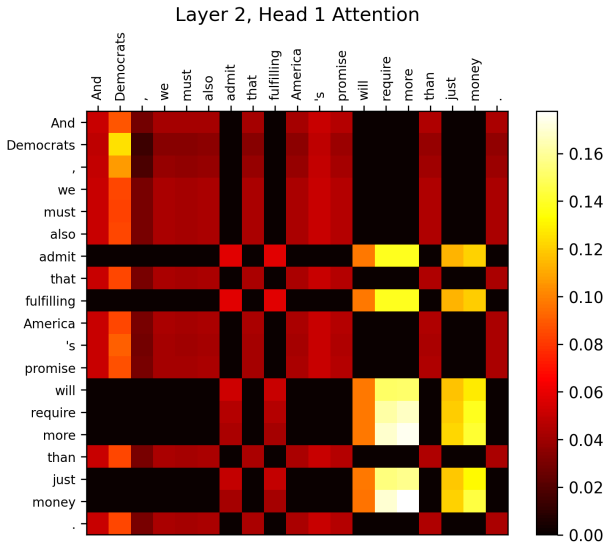
1.4 Sanity Checks

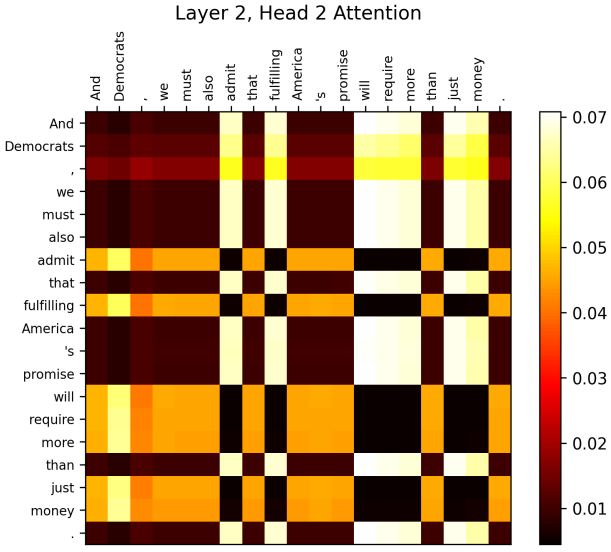
Using the sentence *"*And Democrats, we must also admit that fulfilling America's promise will require more than just money.*"* as an example, we can visualize the attention matrix for a single head in a specific layer. In the plot, columns represent the words being attended to, while rows represent the words attending to others. While certain patterns are observable, many attention weights remain hard to interpret.



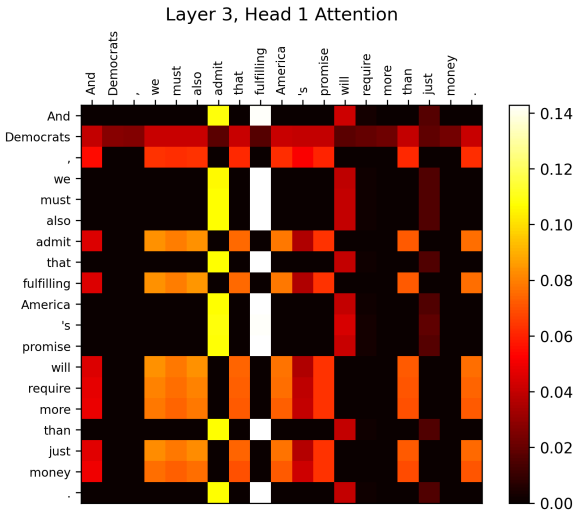


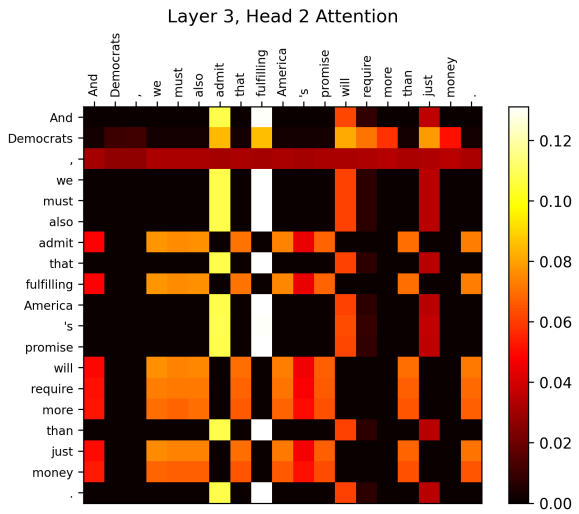
In Layer 1, Head 1, many tokens attend to "must", "also", "'s", and "promise", which means early-layer attention heads often focus on syntactic markers or structurally important words rather than deep semantic relationships. In Head 2, more localized patterns appear. "admit" and "fulfilling" both attend to "Democrats", while "we" and "America" attend to "promise", possibly reflecting semantic associations. However, the attention patterns of "must" and "'s" are harder to interpret, since they do not have a clear relationship.



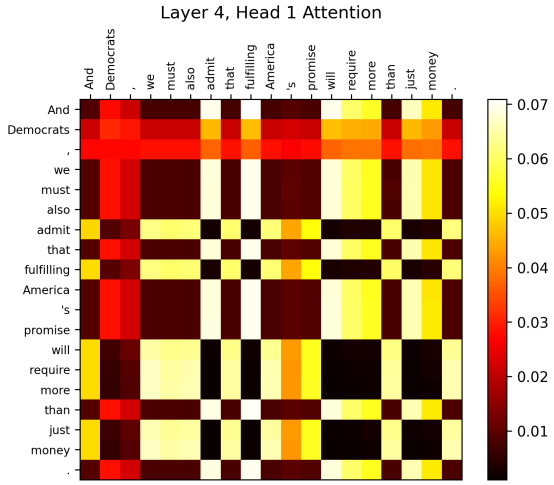


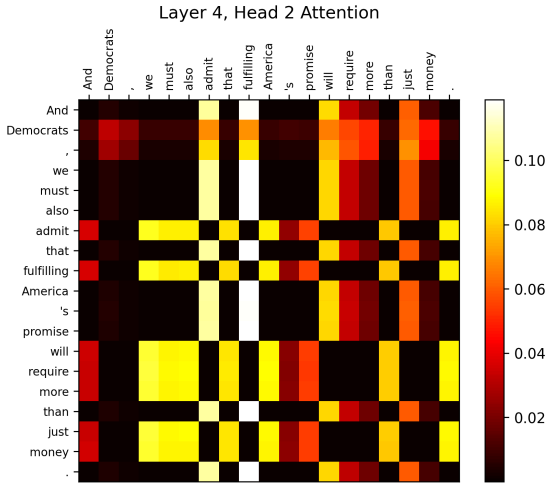
In Layer 2, Head 1 mainly focuses on local context. In contrast, Head 2 shows weaker self-attention (a darker diagonal) and places more weight on distant tokens, indicating longer-distance interactions.
Also, starting from Layer 2, the attention scores become smaller. The range decreases from approximately 0–0.5 in Layer 1 to below 0.2. This suggests that attention becomes more evenly distributed across tokens, rather than being concentrated on a few words.





In Layer 3, the attention patterns of Head 1 and Head 2 are almost identical, which could result from functional redundancy in the model or insufficient training. At the same time, certain tokens, like “we”, “must”, “also”, “admit”, and “fulfilling”, receive much of the attention.



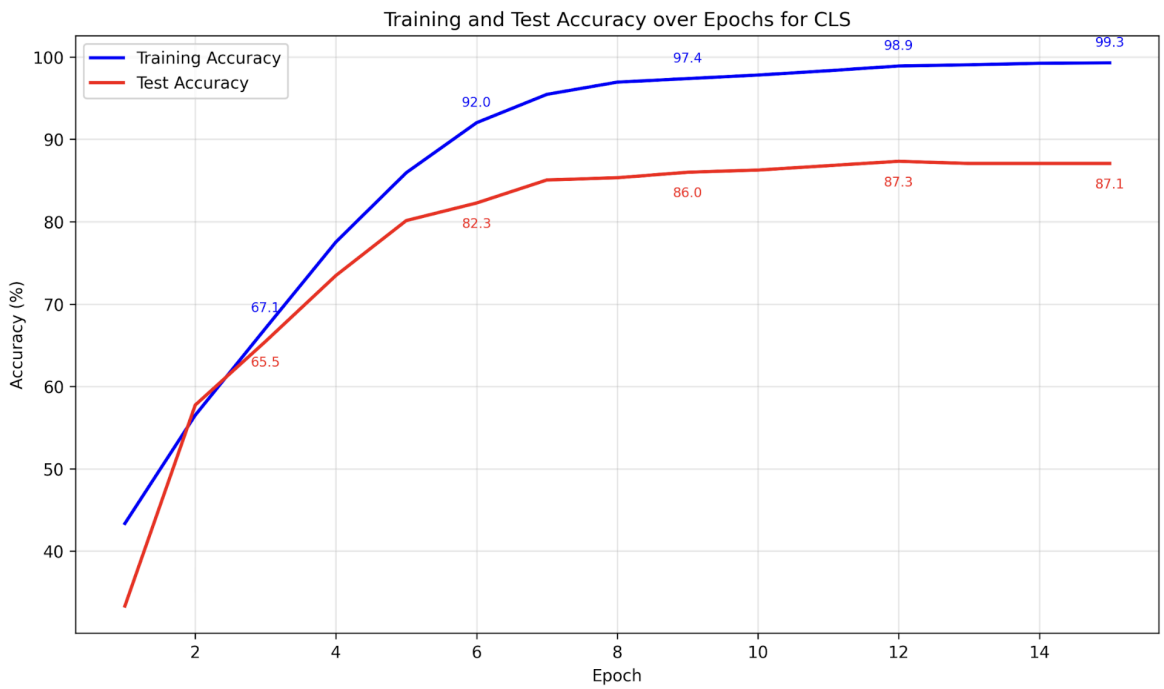


In Layer 4, both heads exhibit weak diagonal attention, indicating reduced focus on local positional information. Head 1 concentrates primarily on "Democrats," the thematic anchor of the sentence. Head 2 focuses on the core predicates "admit" and "fulfilling." This functional specialization reveals a division of labor: Head 1 captures topic structure, while Head 2 tracks predicate-argument relationships.

1.5 Evaluation

The training and test accuracies are shown below. The training accuracy increases from 43% to 99%, while the test accuracy improves from 33% to over 87%. After around epoch 11, the training accuracy continues to increase toward 100%, while the highest test accuracy (87.33%) is observed at epoch 12, after which performance slightly declines. Therefore, early stopping could be applied to prevent further overfitting.

Epoch	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
Train(%)	43.36	56.50	67.07	77.53	85.95	92.02	95.46	96.94	97.37	97.80	98.33	98.90	99.04	99.24	99.28
Test(%)	33.33	57.73	65.47	73.47	80.13	82.27	85.07	85.33	86.00	86.27	86.80	**87.33**	87.07	87.07	87.07



The number of parameters used in this model is summarized in the table below. Based on the model configuration, we can estimate the parameter count as follows. For the embedding layers, the token embedding contains vocab_size * n_embd parameters,and the positional embedding contains block_size * n_embd parameters.Therefore, the total number of parameters in the embedding layers is approximately 370,368. In the multi-head attention module, the query, key, and value projection matrices together contribute 3 * n_embd^2 parameters, while the output projection adds another n_embd^2, and the FFN contains two linear layers with a total of 2 * (n_embd * 4n_embd) = 8 * n_embd^2 parameters. Therefore, the total number of parameters in the encoder is 12 * n_embd^2. Ignoring the relatively small number of parameters in the normalization layers, each Transformer block therefore contains approximately 12 * n_embd^2 parameters. With n_layer = 4, n_embd = 64, the total number of parameters in the Transformer encoder is estimated to be 196608 + 370368 = 566,976. The actual number of encoder parameters is 569,408, which is very close to this theoretical estimation.

Encoder	Classifier	Total
569408	6803	576211

Part 2

2.1 Decoder Implementation

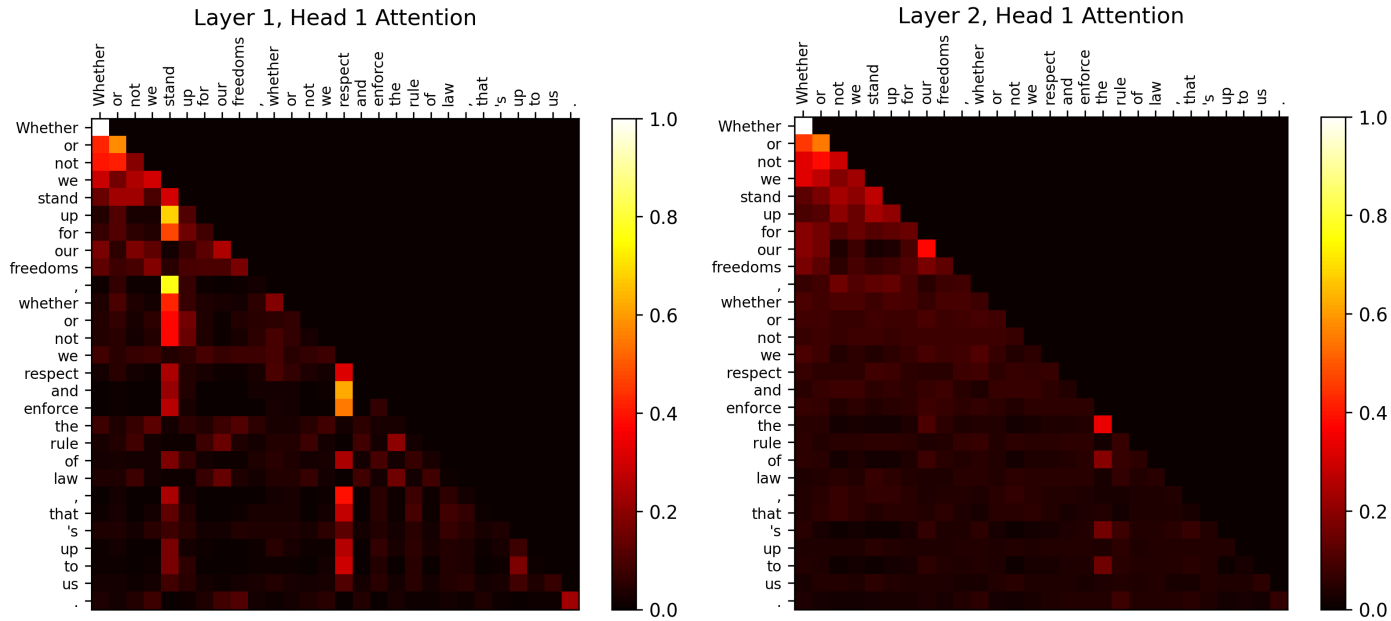
The key distinction from the encoder is the mask applied in the attention, an upper triangular matrix that masks out future positions by setting their attention scores to -inf before softmax. The architecture consists of a token embedding layer and a positional embedding layer, both with dimension n_embd=64. The decoder comprises 4 stacked DecoderBlock layers, each containing a masked multi-head self-attention with 2 attention heads and a feedforward network with a hidden dimension of 100 (using ReLU activation) and a dropout layer(0.3). Each decoder block follows the pre-normalization scheme with layer normalization applied before the attention and

feedforward sub-layers, followed by residual connections. After the final normalization, a linear layer projects the decoder outputs to the vocabulary size, producing next-token prediction. The decoder returns either the cross-entropy loss when targets are provided during training, or the logits and attention weights during inference.

2.2 Decoder Pretraining

The model is trained using the AdamW optimizer with a learning rate of 1e-3 for a maximum of 500 iterations. During training, the model's performance is monitored every 100 iterations by computing perplexity on both the training set and three test sets.To estimate the performance on the test dataset after every 100 training iterations, we evaluate only 100 batches to reduce computational cost. Since perplexity is computed from the average cross-entropy loss, using a subset of the data provides a sufficiently accurate approximation.

2.3 Sanity Check

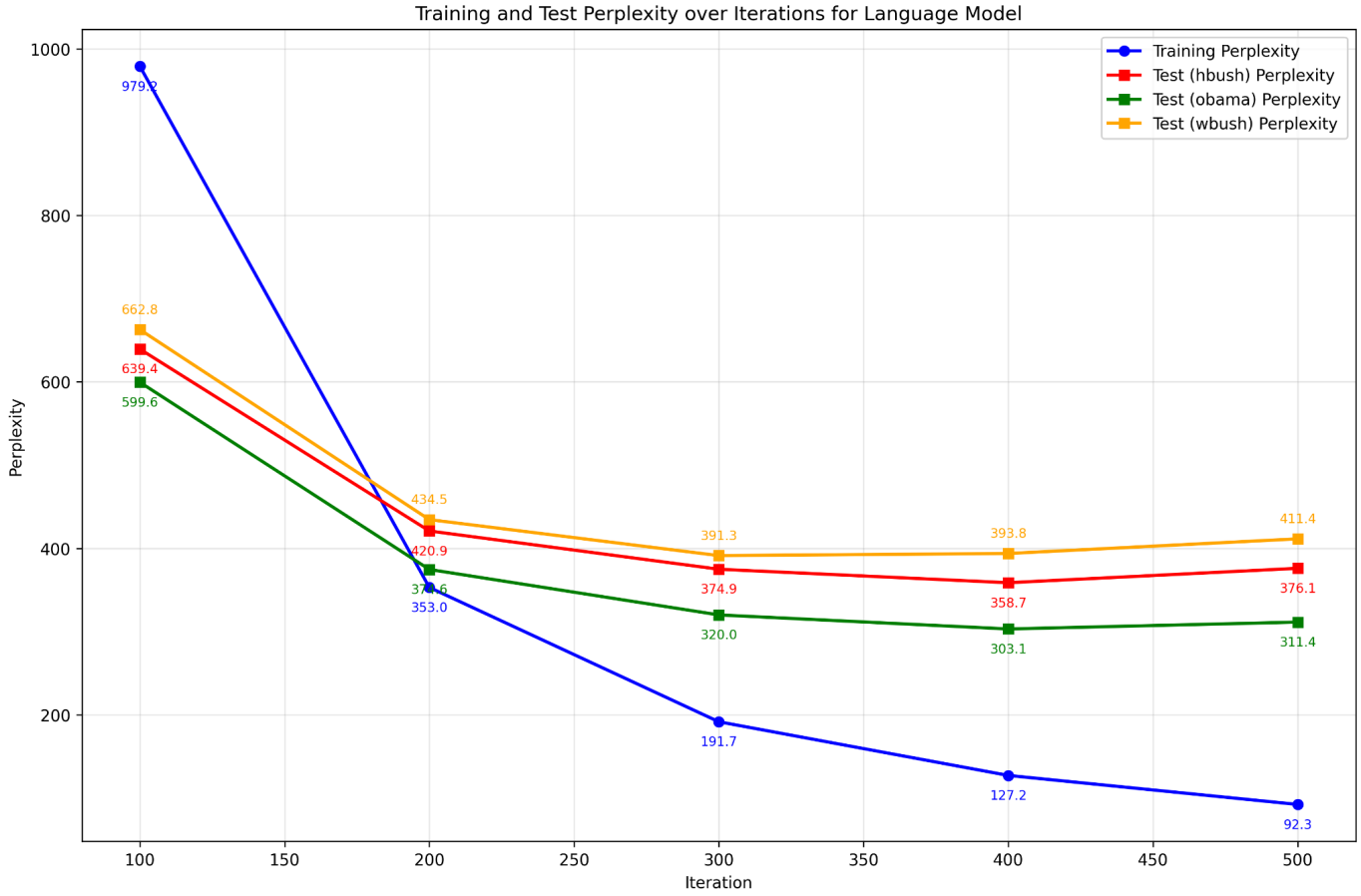


Using the sentence “Whether or not we stand up for our freedoms, whether or not we respect and enforce the rule of law, that's up to us.” as an example, we can visualize the attention matrix for a single head in a specific layer. Interpreting the results comprehensively is challenging, so I selected a few heads that show some patterns for analysis. First, in Layer 1, Head 1, the diagonal is prominently bright, which indicates a strong local focus. Additionally, I noticed that many tokens significantly attend to the verbs 'stand' and 'respect'. It makes sense because verbs are crucial for understanding the structure and meaning of each sentence. In Layer 2, Head 1, the word 'the' receives high attention from other tokens This indicates other parts of the sentence pay attention to it to resolve grammatical structure, since the word 'the' signals that a noun is coming
Also, the attention weights for the tokens at the beginning of the sentence (the top rows) are brighter, while those at the end (the bottom rows) are dimmer. This is a consequence of the fact that attention weights for each token must sum to 1. Tokens at the beginning have a limited context to look at, so they must assign higher weights to visible tokens. Tokens at the end have a long context, so their attention is spread across many words.

2.4 Evaluation

The learning curve demonstrates effective training progress, with training perplexity decreasing from 979 at step 100 to 92 after 500 iterations, which indicates successful learning of language patterns. However, a gap exists between the final training perplexity (92.3) and test perplexities (311.4-411.4). Moreover, test perplexities

begin to increase after 400 iterations, suggesting overfitting. The final model achieves test perplexities of **371.58 on H. Bush**, **311.48 on Obama**, and **458.98 on W. Bush**.
The superior performance on Obama's speeches and poorer results on W. Bush's corpus can be attributed to two main factors. First, the training dataset may contain an imbalanced distribution of speech samples from different political figures. Second, Obama's speech style tends to follow more predictable, structured patterns, making his speeches easier for language models to predict.



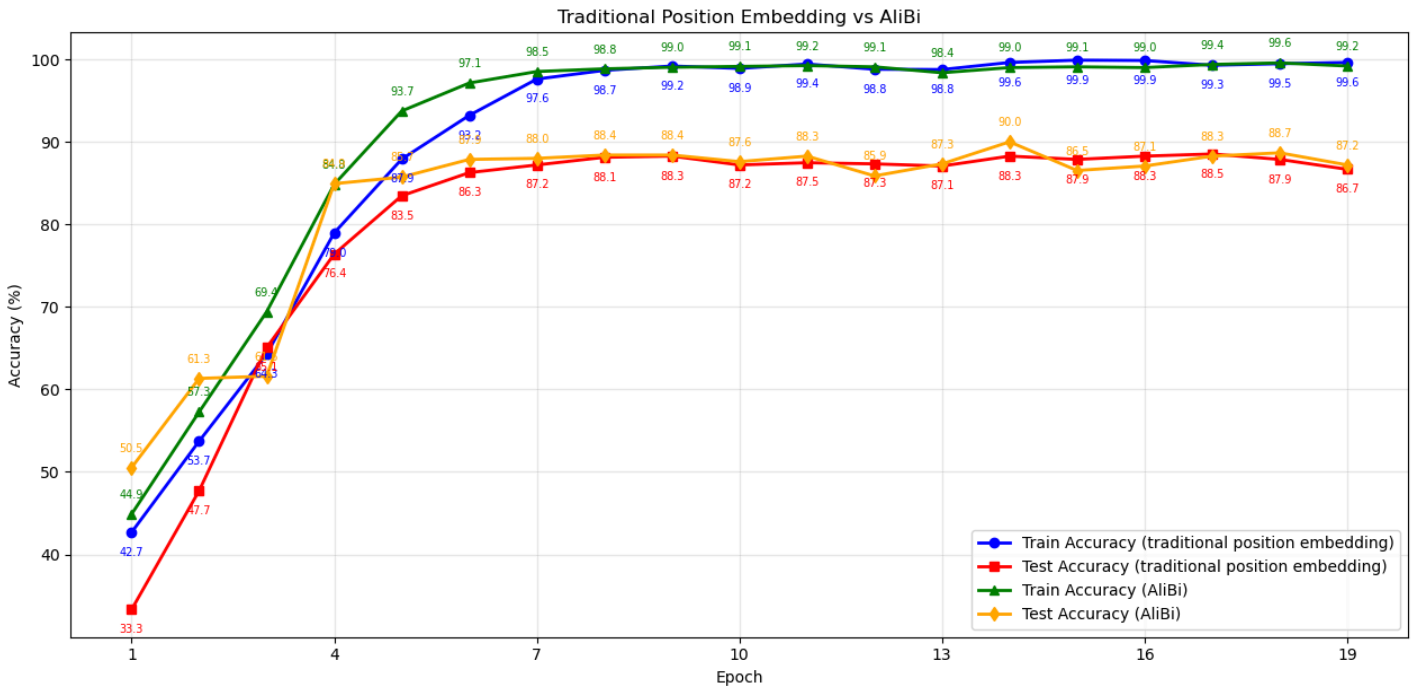
The number of parameters used in this model is **862,987**. Based on the model configuration, we can estimate the parameter count as follows. For the embedding layers, the token embedding contains vocab_size * n_embd parameters,and the positional embedding contains block_size * n_embd parameters.Therefore, the total number of parameters in the embedding layers is approximately 370,368. In the multi-head attention module, the query, key, and value projection matrices together contribute 3 * n_embd^2 parameters, while the output projection adds another n_embd^2, and the FFN contains two linear layers with a total of 2 * (n_embd *n_hidden) = 12,800 parameters. Therefore, the total number of parameters in the 4 attention layers is (12,800 + 4 * 64 * 64) * 4 = 116,736. The LM head layer has 368,320 parameters. Ignoring the relatively small number of parameters in the normalization layers, the total number of parameters in the Transformer decoder is estimated to be 370,368 + 116,736 + 368,320 = 855,424. The actual number of decoder parameters is 862,987, which is very close to this theoretical estimation.

Part 3

I experimented with positional encoding by replacing the traditional learned positional embeddings with ALiBi. While keeping all other hyperparameters identical, I compared the performance of the tasks from Part 1 and Part 2 using both methods.The ALiBi implementation is applied directly to the attention scores. For each attention head, I compute a slope and construct a distance matrix that represents the absolute difference

between two token positions. The bias for each head is $bias = -slop * |distance|$. This bias is added to the attention scores before the softmax. Now, the attention scores is $\frac{QK^T}{\sqrt{d}} + bias$. As a result, the model favors closer tokens without relying on learned positional embeddings. When ALiBi is enabled, token embeddings are used alone, and positional embeddings are not added.

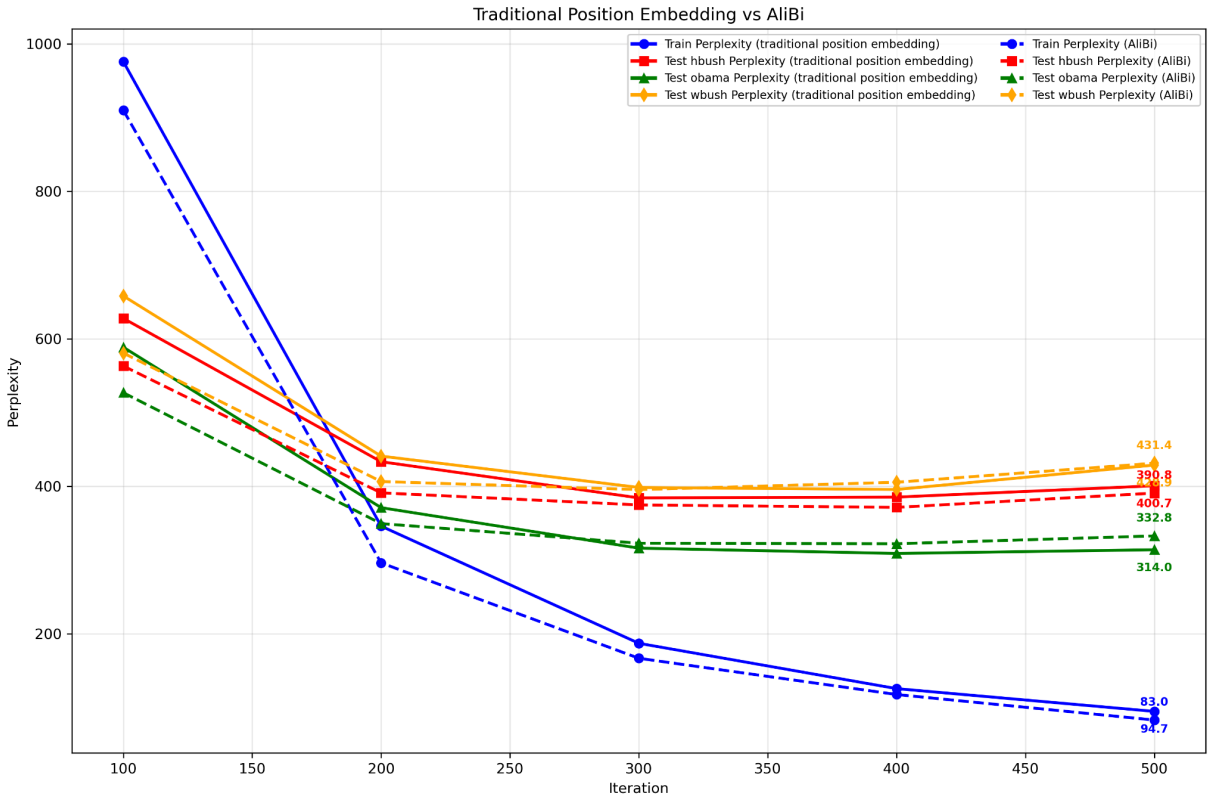
3.1 Results on Encoder + Classifier



The results of the classification task are shown above. We observe that the model with ALiBi converges earlier compared to the model with traditional positional embeddings. However, their final training and test accuracies are similar. A likely explanation is that in this specific task, we use an average network, which aggregates information across all tokens. This aggregation may largely ignore word order, meaning that positional information does not significantly impact the final classification result.

3.2 Results on Decoder

Based on the training logs, both models are learning effectively, but the AliBi model converges a bit faster and achieves a slightly lower final training perplexity. The improvement from AliBi is very modest and not consistent across all test datasets. For example, for hbush, test perplexity decreased slightly from 400.7 to 390.8, which is a small improvement. For obama, it actually increased from 313.99 to 332.77, and for wbush, it also increased a bit from 428.86 to 431.38. So the model with AliBi performs slightly worse on the test set, even though the training perplexity is lower. This suggests that while AliBi helps the model fit the training data more efficiently, it does not consistently improve generalization on these test datasets. The benefit of AliBi might be more noticeable for longer sequences or larger datasets.


Traditional Position Embedding vs AliBi
Train Perplexity (traditional position embedding)
Test hbush Perplexity (traditional position embedding)
Test obama Perplexity (traditional position embedding)
Test wbush Perplexity (traditional position embedding)
Train Perplexity (AliBi)
Test hbush Perplexity (AliBi)
Test obama Perplexity (AliBi)
Test wbush Perplexity (AliBi)
Perplexity
Iteration
1000
800
600
400
200
100
150
200
250
300
350
400
450
500
431.4
390.8
400.7
332.8
314.0
83.0
94.7